Lab02. Running Docker container within Cloud9

Creation and Administration of Cloud-based Applications

Wojciech Thomas

Fall 2025

1 Learning objectives

1. Manage containers using CLI
2. Build and run own container with simple Flask application

2 Exercises

2.1 Start

1. Start your AWS Learning Lab, and open AWS Console.
2. Create Cloud9 development environment. Select:
   - use new EC2 instance,
   - connection by SSH.

2.2 Creating simple Flask project

1. Create a new folder `demo` in the `~/environment/` folder.
2. In the folder `demo` create a new file `requirements.txt`. Put the following content inside:

   ```
   flask
   ```

3. Create a new file `app.py`. Put the following content inside:

   ```
   from flask import Flask

   app = Flask(__name__)

   @app.route("/")
   def hello_world():
     return "<p>Hello, World!</p>"

   if __name__ == "__main__":
     app.run(host="0.0.0.0", debug=True)
   ```

4. Configure virtual Python environment for testing your application by hand. Open Terminal window in Cloud9 IDE (menu Window > New Terminal), then type following commands:

   ```
   $ cd ~/environment/demo
   $ python3 -m venv .venv
   $ source ./.venv/bin/activate
   (.env)$ pip install -r requirements.txt
   ```

```
(.env)$ python app.py
```

You should see running application.

5. Open **new** Terminal tab in Cloud9, and test if your application answers web requests:

```
curl http://localhost:5000/
```

6. Find the EC2 instance running Cloud9 environment. Change its Security Group to allow *Inbound access* to port TCP 5000 from any IPv4 address.
7. Find the public DNS address of Cloud9 EC2 instance.
8. Open any browser and type the address: `http://<public-dns-address-of-cloud9>:5000/`. You should see the "Hello world"
9. Stop the Flask application using Ctrl+C.

2.3 Build the container image

1. In folder `demo` (next to `app.py` file) create a new file named `Dockerfile`
2. Type the following content inside:

```
FROM python:3.8-alpine
COPY ./requirements.txt /app/requirements.txt
WORKDIR /app
RUN pip install -r requirements.txt
COPY . /app
EXPOSE 5000
ENTRYPOINT [ "python" ]
CMD ["app.py" ]
```

3. Build a new image: `docker build -t myapp:v1 .` (pay attention to the dot!)
4. List all available images: `docker images`. Find your image
5. Run container with the application: `docker run -i -p 5000:5000 myapp:v1`. Option `-i` means *interactive* – your application will run in the foreground, and you will not be able to use this Terminal.
6. Open the other Terminal tab, and check, if you can access the application: `curl http://localhost:50`
7. Refresh the page in your browser. Do you still see the same page?
8. Get back to the first terminal and stop the app by pressing Ctrl+C.
9. Run a container again: `docker run -d -p 5000:5000 myapp:v1`

   Does it run the same way as previously? Can you run the `curl` command in this Terminal tab?
10. As your application runs in the background, you cannot stop it using Ctrl-C. List running containers: `docker ps`. Find your container. Stop it, using command: `docker stop 34ea1` (remember to replace number with the ID of your container).

2.4 Rebuild the container image

1. Modify slightly the Flask application (e.g. replace the welcome message).
2. Build the new container image using command: `docker build -t myapp:v2 .`
3. Run the new version of your application.
4. Test the output from the new application.

5. Stop and remove the container.
6. Run in the terminal commands:

```
docker run -d -p 5001:5000 myapp:v1
docker run -d -p 5002:5000 myapp:v2
docker ps
curl http://localhost:5001/
curl http://localhost:5002/
```